

Chapter 19. Functional programming techniques

This chapter covers

- First-class citizens, higher-order functions, currying, and partial application
- Persistent data structures
- Lazy evaluation and lazy lists as generalizing Java streams
- Pattern matching and how to simulate it in Java
- Referential transparency and caching

In [chapter 18](#), you saw how to think functionally; thinking in terms of side-effect-free methods can help you write more maintainable code. In this chapter, we introduce more advanced functional programming techniques. You can view this chapter as being a mix of practical techniques to apply in your code base, as well as academic information that will make you a more knowledgeable programmer. We discuss higher-order functions, currying, persistent data structures, lazy lists, pattern matching, caching with referential transparency, and combinators.

19.1. Functions everywhere

In [chapter 18](#) we used the phrase *functional-style programming* to mean that the behavior of functions and methods should be like that of mathematical-style functions, with no side effects. Functional-language programmers often use the phrase with more generality to mean that functions may be used like other values: passed as arguments, returned as results, and stored in data structures. Functions that may be used like other values are referred to as *first-class functions*. First-class functions are what Java 8 added over previous versions of Java: you may use any method as a function value, using the `::` operator to create a method reference, and lambda expressions (such as `(int x) -> x + 1`) to express function values directly. In Java 8 it's perfectly valid^[1] to store the method `Integer.parseInt` in a variable by using a method reference as follows:

Other formats available

 Audiobook 

If `Integer::parseInt` is the only method you plan to store in variable `strToInt`, you might want to make `strToInt` have type `ToIntFunction<String>` to save boxing. You don't do so here because using such Java primitive applications can get in the way of

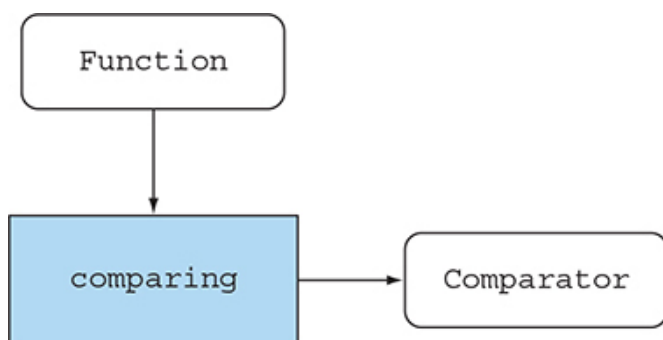
seeing what's happening, even if those applications improve efficiency for primitive types.

```
Function<String, Integer> strToInt = Integer::parseInt;
```

19.1.1. Higher-order functions

So far, you've mainly used the fact that function values are first-class only to pass them to Java 8 stream-processing operations (as in [chapters 4–7](#)) and to achieve the similar effect of behavior parameterization when you passed `Apple::isGreen` as a function value to `filterApples` in [chapters 1](#) and [2](#). Another interesting example was using the static method `Comparator.comparing`, which takes a function as a parameter and returns another function (a `Comparator`), as illustrated in the following code and [figure 19.1](#):

Figure 19.1. `comparing` takes a function as parameter and returns another function.



```
Comparator<Apple> c = comparing(Apple::getWeight);
```

You did something similar when you composed functions in [chapter 3](#) to create a pipeline of operations:

```
Function<String, String> transformationPipeline  
    = addHeader.andThen(Letter::checkSpelling)  
        .andThen(Letter::addFooter);
```

Functions (such as `Comparator.comparing`) that can do at least one of

Other formats available



- Take one or more functions as a parameter
- Return a function as a result

Higher-order functions within the functional pro-

This characterization directly relates to Java 8 functions because they can not only be passed as arguments, but also returned as results, assigned to local variables, or even inserted into structures. A pocket-calculator program might have a `Map<String, Function<Double, Double>>` that maps the `String "sin"` to `Function<Double, Double>` to hold the method reference `Math::sin`. You did something similar when you learned about the factory design pattern in [chapter 8](#).

Readers who liked the calculus example at the end of [chapter 3](#) can regard the type of differentiation as being

```
Function<Function<Double,Double>, Function<Double,Double>>
```

because it takes a function as an argument (such as `(Double x) -> x * x`) and returns a function as a result (in this example, `(Double x) -> 2 * x`). We've written this code as a function type (the leftmost `Function`) to explicitly affirm the fact that you could pass this differentiating function to another function. But it's good to recall that the type for differentiating and the signature

```
Function<Double,Double> differentiate(Function<Double,Double> func)
```

say the same thing.

Side effects and higher-order functions

We noted in [chapter 7](#) that functions passed to stream operations generally are side-effect-free, and we noted the problems that arise otherwise (such as incorrect results and even unpredictable results due to race conditions you hadn't thought of). This principle also applies in general when you use higher-order functions. When you're writing a higher-order function or method, you don't know in advance what arguments will be passed to it and, if the arguments have side effects, what these side effects might do. It becomes far too complicated to reason about what your code does if it uses functions passed as arguments that make unpredictable changes in the state of your program; such functions might even interfere

Other formats available



Audiobook



to-debug way. It's a good design principle to be willing to accept from functions passed all!

In the next section, we turn to currying: a technique that can help you modularize functions and reuse code.

19.1.2. Currying

Before we give you the theoretical definition of currying, we'll present an example. Applications almost always need to be internationalized, so converting from one set of units to another set is a problem that comes up repeatedly.

Unit conversion always involves a conversion factor and, from time to time, a baseline adjustment factor. The formula to convert Celsius to Fahrenheit, for example, is $CtoF(x) = x * 9/5 + 32$. The basic pattern of all unit conversion is as follows:

1. Multiply by the conversion factor.
2. Adjust the baseline if relevant.

You can express this pattern with the following general method:

```
static double converter(double x, double f, double b) {  
    return x * f + b;  
}
```

Here, x is the quantity you want to convert, f is the conversion factor, and b is the baseline. But this method is a bit too general. Typically, you require a lot of conversions between the same pair of units, such as kilometers to miles. You could call the `converter` method with three arguments on each occasion, but supplying the factor and baseline each time would be tedious, and you might accidentally mistype them.

You could write a new method for each application, but doing so would miss the reuse of the underlying logic.

Here's an easy way to benefit from the existing logic while tailoring the converter for particular applications. You can define a factory that manufactures one-argument conversion functions to exemplify the idea of currying:

Other formats available



Audiobook



```
    static CurriedConverter curriedConverter(double f, double b){  
        return x -> x * f + b;  
    }
```

Now all you have to do is pass `curriedConverter` the conversion factor and baseline (f and b), and it obligingly returns a function (of x) to do

what you asked for. Then you can use the factory to produce any converter you require, as follows:

```
DoubleUnaryOperator convertCtoF = curriedConverter(9.0/5, 32);
DoubleUnaryOperator convertUSDtoGBP = curriedConverter(0.6, 0);
DoubleUnaryOperator convertKmttoMi = curriedConverter(0.6214, 0);
```

Because `DoubleUnaryOperator` defines a method `applyAsDouble`, you can use your converters as follows:

```
double gbp = convertUSDtoGBP.applyAsDouble(1000);
```

As a result, your code is more flexible, and it reuses the existing conversion logic!

Reflect on what you're doing here. Instead of passing all the arguments x , f , and b all at once to the `converter` method, you ask only for the arguments f and b and return another function—which, when given an argument x , returns $x * f + b$. This two-stage process enables you to reuse the conversion logic and create different functions with different conversion factors.

Formal definition of currying

Currying^{[a](#)} is a technique in which a function f of two arguments (such as x and y) is instead viewed as a function g of one argument that returns a function also of one argument. The value returned by the latter function is the same as the value of the original function—that is, $f(x, y) = (g(x))(y)$.

^{[a](#)}

The word currying is unconnected to Indian food; the term is named after the logician Haskell Brooks Curry, who popularized the technique. He attributed it to Moses Ilyich Schönfinkel, however. Should we refer to currying as schönfinkeling instead?

Other formats available



guments, 1 and 3.



course. You can curry a six-argument function numbered 2, 4, and 6, which returns a function that returns a function taking the remaining ar-

When some arguments (but fewer than the full complement of arguments) have been passed, the function is *partially applied*.

In the next section, we turn to another aspect of functional-style programming: data structures. Is it possible to program with data structures if you're forbidden to modify them?

19.2. Persistent data structures

Data structures used in functional-style programs have various names, such as functional data structures and immutable data structures, but perhaps the most common is persistent data structures. (Unfortunately, this terminology clashes with the notion of *persistent* in databases, meaning “outliving one run of the program.”)

The first thing to notice is that a functional-style method isn't allowed to update any global data structure or any structure passed as a parameter. Why? Because calling it twice is likely to produce different answers, violating referential transparency and the ability to understand the method as a simple mapping from arguments to results.

19.2.1. Destructive updates vs. functional

Consider the problems that can arise. Suppose that you represent train journeys from A to B as a mutable `TrainJourney` class (a simple implementation of a singly linked list), with an `int` field modeling some detail of the journey, such as the price of the current leg of the journey. Journeys that require changing trains have several linked `TrainJourney` objects via the `onward` field; a direct train or the final leg of a journey has `onward` being `null`:

```
class TrainJourney {
    public int price;
    public TrainJourney onward;
    public TrainJourney(int p, TrainJourney t) {
        price = p;
        onward = t;
    }
}
```

Other formats available



Audiobook



Separate `TrainJourney` objects representing
n Y to Z. You may want to create one jour-
ney objects (that is, X to Y to Z).

Here is a simple traditional imperative method to link these train journeys:

```

static TrainJourney link(TrainJourney a, TrainJourney b){
    if (a==null) return b;
    TrainJourney t = a;
    while(t.onward != null){
        t = t.onward;
    }
    t.onward = b;
    return a;
}

```

This method works by finding the last leg in the `TrainJourney` for `a` and replacing the `null` marking the end of `a`'s list with list `b`. (You need a special case if `a` has no elements.)

Here's the problem: suppose that a variable `firstJourney` contains the route from X to Y and that a variable `secondJourney` contains the route from Y to Z. If you call `link(firstJourney, secondJourney)`, this code destructively updates `firstJourney` to also contain `secondJourney`, so in addition to the single user who requests a trip from X to Z seeing the combined journey as intended, the journey from X to Y has been updated destructively. Indeed, the `firstJourney` variable is no longer a route from X to Y, but one from X to Z, which breaks code that depends on `firstJourney`'s not being modified! Suppose that `firstJourney` represented the early-morning London-to-Brussels train, which all subsequent users trying to get to Brussels will be surprised to see requires an onward leg, perhaps to Cologne. We've all fought battles with such bugs concerning how visible a change in a data structure should be.

The functional-style approach to this problem is to ban such side-effecting methods. If you need a data structure to represent the result of a computation, you should make a new one, not mutate an existing data structure, as you've done previously. Doing so is often a best practice in standard object-oriented programming, too. A common objection to the functional approach is that it causes excess copying, and the programmer says, "I'll remember" or "I'll document" the side effects. But such optimism is a trap for maintenance programmers who have to deal with your code later. Thus, the functional-style solution is as follows:

Other formats available



Audiobook



```

append(TrainJourney a, TrainJourney b){
    b : new TrainJourney(a.price, append(a.onward, b)

```

This code is clearly functional-style (uses no mutation, even locally) and doesn't modify any existing data structures. Note, however, that the code doesn't create a new `TrainJourney`. If `a` is a sequence of n elements and

b is a sequence of m elements, the code returns a sequence of $n+m$ elements, in which the first n elements are new nodes and the final m elements share with TrainJourney b. Note that users are required not to mutate the result of append because by doing so, they may corrupt the trains passed as sequence b. [Figures 19.2](#) and [19.3](#) illustrate the difference between the destructive append and the functional-style append.

Figure 19.2. The data structure is destructively updated.

Destructive append

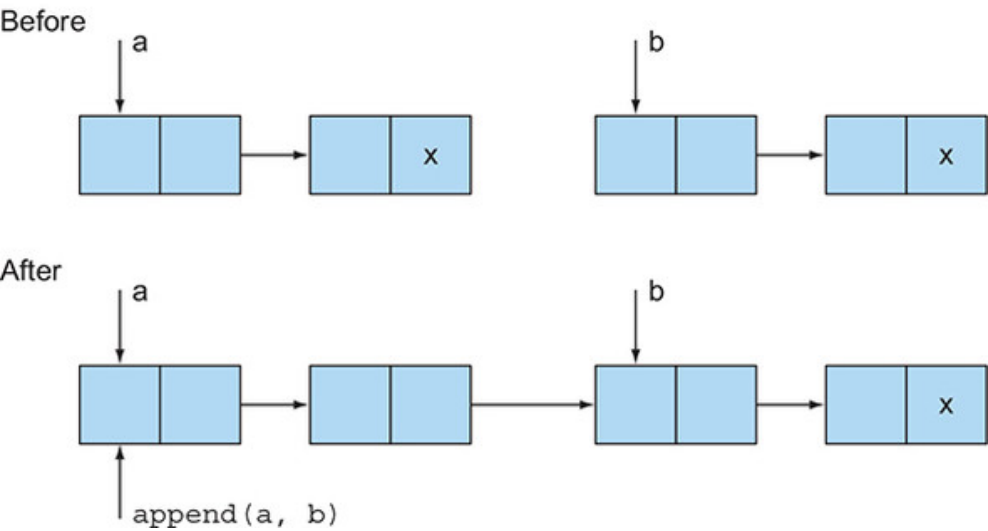
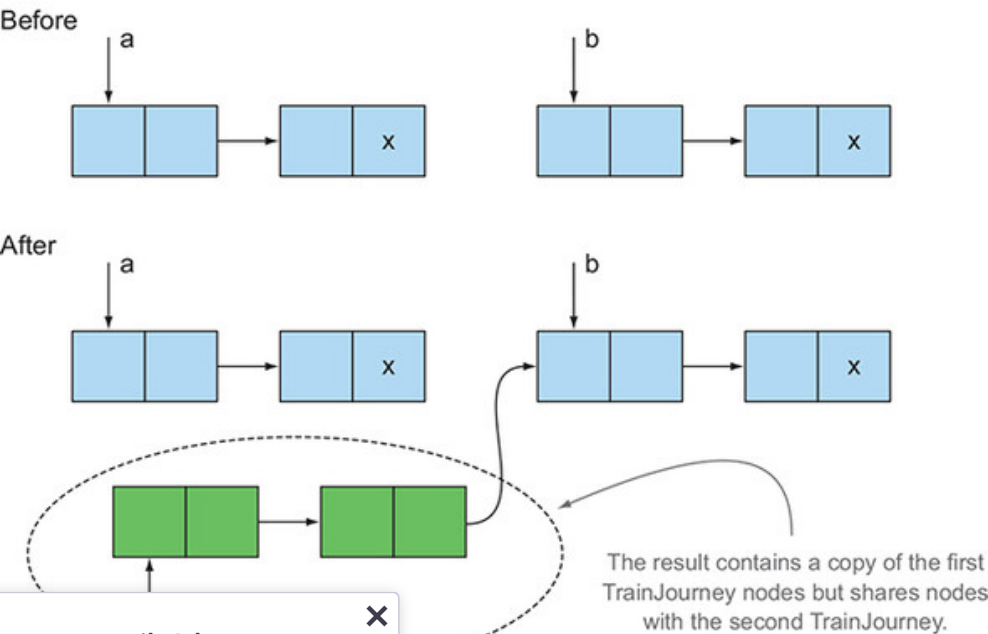


Figure 19.3. Functional style with no modifications to the data structure

Functional-style append



Other formats available



Audiobook



h Trees

Before leaving this topic, consider another data structure: a binary search tree that might be used to implement a similar interface to a HashMap.

The idea is that a `Tree` contains a `String` representing a key and an `int` representing its value, perhaps names and ages:

```
class Tree {
    private String key;
    private int val;
    private Tree left, right;
    public Tree(String k, int v, Tree l, Tree r) {
        key = k; val = v; left = l; right = r;
    }
}

class TreeProcessor {
    public static int lookup(String k, int defaultval, Tree t) {
        if (t == null) return defaultval;
        if (k.equals(t.key)) return t.val;
        return lookup(k, defaultval,
                      k.compareTo(t.key) < 0 ? t.left : t.right);
    }
    // other methods processing a Tree
}
```

You want to use the binary search tree to look up `String` values to produce an `int`. Now consider how you might update the value associated with a given key (assuming for simplicity that the key is already present in the tree):

```
public static void update(String k, int newval, Tree t) {
    if (t == null) { /* should add a new node */ }
    else if (k.equals(t.key)) t.val = newval;
    else update(k, newval, k.compareTo(t.key) < 0 ? t.left : t.right);
}
```

Adding a new node is trickier. The easiest way is to make the method update return the `Tree` that has been traversed (unchanged unless you had to add a node). Now this code is slightly clumsier because the user needs to remember that update tries to update the tree in place, returning the same tree as passed. But if the original tree were empty, a new node is returned as a result:

Other formats available



Audiobook



```
update(String k, int newval, Tree t) {
    if (t == null) return new Tree(k, newval, null, null);
    else if (k.equals(t.key))
        t.val = newval;
    else if (k.compareTo(t.key) < 0)
        t.left = update(k, newval, t.left);
    else
```

```

        t.right = update(k, newval, t.right);
    return t;
}

```

Note that both versions of `update` mutate the existing `Tree`, meaning that all users of the map stored in the tree see the mutation.

19.2.3. Using a functional approach

How might you program such tree updates functionally? You need to create a new node for the new key-value pair. You also need to create new nodes on the path from the root of the tree to the new node, as follows:

```

public static Tree fupdate(String k, int newval, Tree t) {
    return (t == null) ?
        new Tree(k, newval, null, null) :
        k.equals(t.key) ?
            new Tree(k, newval, t.left, t.right) :
        k.compareTo(t.key) < 0 ?
            new Tree(t.key, t.val, fupdate(k, newval, t.left), t.right) :
            new Tree(t.key, t.val, t.left, fupdate(k, newval, t.right));
}

```

In general, this code isn't expensive. If the tree is of depth d and reasonably well balanced, it can have around 2^d entries, so you re-create a small fraction of it.

We've written this code as a single conditional expression instead of using `if-then-else` to emphasize the idea that the body is a single expression with no side effects. But you may prefer to write an equivalent `if-then-else` chain, each containing a `return`.

What's the difference between `update` and `fupdate`? We noted previously that the method `update` assumes that every user wants to share the data structure and see updates caused by any part of the program. Therefore, it's vital (but often overlooked) in nonfunctional code that whenever you add some form of structured value to a tree, you copy it, because someone may assume later that he can update it. By contrast,

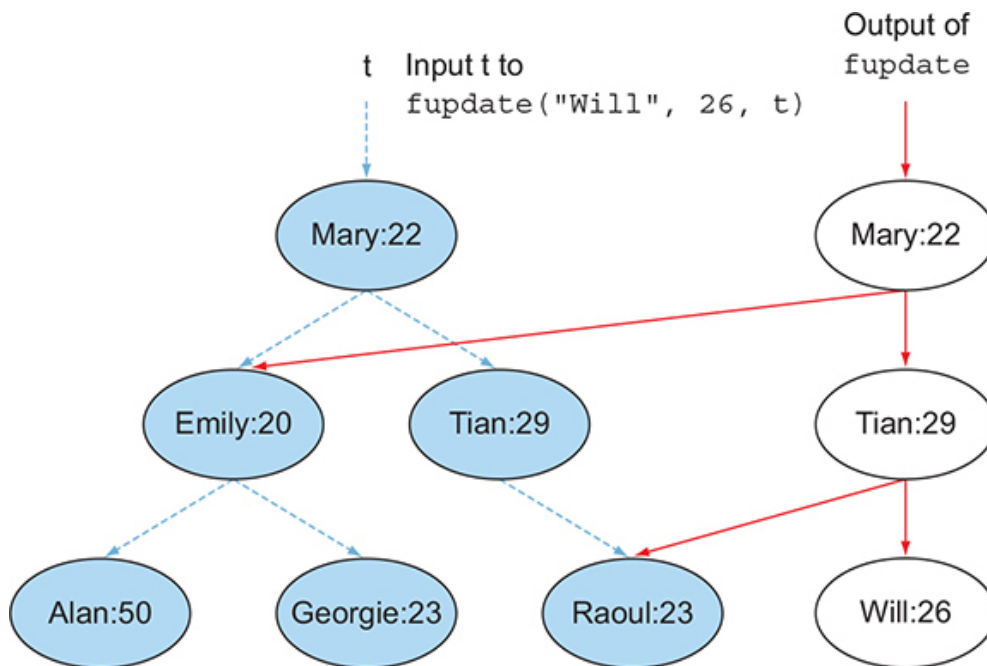
Other formats available

 Audiobook 

✕ it creates a new `Tree` as a result but shares the existing data structure. [Figure 19.4](#) illustrates this idea. You can think of these trees as nodes that store a name and an age of a person. In `update`, you mutate the existing tree; it creates new nodes

“living at the side of” the tree without harming the existing data structure.

Figure 19.4. No existing data structure was harmed during the making of this update to Tree.



Such functional data structures are often called *persistent*—their values persist and are isolated from changes happening elsewhere—so as a programmer, you’re sure that `fupdate` won’t mutate the data structures passed as its arguments. There’s one proviso: the other side of the treaty requires all users of persistent data structures to follow the do-not-mutate requirement. If not, a programmer who disregards this proviso might mutate the result of `fupdate` (by changing Emily’s 20, for example). Then this mutation would be visible as an (almost certainly unwanted) unexpected and delayed change to the data structure passed as argument to `fupdate`!

Viewed in these terms, `fupdate` can be more efficient. The “no mutation of existing structure” rule allows structures that differ only slightly (such as the Tree seen by user A and the modified version seen by user B) to share storage for common parts of their structure. You can get the compiler to help enforce this rule by declaring fields `key`, `val`, `left`, and `right` of class `Tree` to be `final`. But remember that `final` protects only a field, not the object pointed to, which may need its own fields to be `final` to protect it, and so on.

You might say, “I want to update the tree to be seen by some users (but not others).” You have two choices. One choice is the imperative-style solution: you logically make a new data structure whenever you do an update (so that nothing is ever mutated) and arrange to pass the correct version of the data structure to users as appropriate. This idea could be enforced through an API. If certain clients of the data structure need to

Other formats available



Audiobook

have updates visible, they should go through an API that returns the latest version. Clients who don't want updates visible (such as for long-running statistical analysis) use whatever copy they retrieve, knowing that it can't be mutated from under them.

This technique is like updating a file on a CD-R, which allows a file to be written only once by burning with a laser. Multiple versions of the file are stored on the CD (smart CD-authoring software might even share common parts of multiple versions), and you pass the appropriate block address of the start of the file (or a filename encoding the version within its name) to select which version you want to use. In Java, things are rather better than on a CD, in that old versions of the data structure that can no longer be used are garbage-collected.

19.3. Lazy evaluation with streams

You saw in previous chapters that streams are great ways to process a collection of data. But for various reasons, including efficient implementation, the Java 8 designers added streams to Java in a rather specific way. One limitation is that you can't define a stream recursively because a stream can be consumed only once. In the following sections, we show you how this situation can be problematic.

19.3.1. Self-defining stream

Revisit the example from [chapter 6](#) of generating prime numbers to understand this idea of a recursive stream. In that chapter, you saw that (perhaps as part of a `MyMathUtils` class), you can compute a stream of prime numbers as follows:

```
public static Stream<Integer> primes(int n) {
    return Stream.iterate(2, i -> i + 1)
        .filter(MyMathUtils::isPrime)
        .limit(n);
}

public static boolean isPrime(int candidate) {
    int candidateRoot = (int) Math.sqrt((double) candidate);
    return IntStream.rangeClosed(2, candidateRoot)
        .noneMatch(i -> candidate % i == 0);
}
```

Other formats available



awkward. You have to iterate through every number every time to see whether it can be divided by a candidate number. (In fact, you need only test numbers that have been already classified as prime.)

Ideally, the stream should filter out numbers that are divisible by the prime that the stream is producing on the go. Here's how this process might work:

1. You need a stream of numbers from which you'll select prime numbers.
2. From that stream, take the first number (the head of the stream), which will be a prime number. (In the initial step, this number is 2.)
3. Filter all the numbers that are divisible by that number from the tail of the stream.
4. The resulting tail is the new stream of numbers that you can use to find prime numbers. Essentially, you go back to step 1, so this algorithm is recursive.

Note that this algorithm is poor for a few reasons,^[2] but it's simple to reason about algorithms for the purpose of working with streams. In the following sections, you try to write this algorithm by using the Streams API.

[2](#)

You can find more information about why the algorithm is poor at www.cs.hmc.edu/~oneill/papers/Sieve-JFP.pdf

Step 1: Get a stream of numbers

You can get an infinite stream of numbers starting from 2 by using the method `IntStream.iterate` (which we described in [chapter 5](#)) as follows:

```
static IntStream numbers(){
    return IntStream.iterate(2, n -> n + 1);
}
```

Step 2: Take the head

An `IntStream` comes with the method `findFirst`, which you can use to return the first element:

```
static int head(IntStream numbers){
    return numbers.findFirst().getAsInt();
}
```

Other formats available

 Audiobook 

Step 3: Filter the tail

Define a method to get the tail of a stream:

```
static IntStream tail(IntStream numbers){
    return numbers.skip(1);
}
```

Given the head of the stream, you can filter the numbers as follows:

```
IntStream numbers = numbers();
int head = head(numbers);
IntStream filtered = tail(numbers).filter(n -> n % head != 0);
```

Step 4: Recursively create a stream of primes

Here comes the tricky part. You may be tempted to try passing back the resulting filtered stream so that you can take its head and filter more numbers, like this:

```
static IntStream primes(IntStream numbers) {
    int head = head(numbers);
    return IntStream.concat(
        IntStream.of(head),
        primes(tail(numbers).filter(n -> n % head != 0))
    );
}
```

Bad news

Unfortunately, if you run the code in step 4, you get the following error:
`java.lang.IllegalStateException: stream has already been operated upon or closed.` Indeed, you're using two terminal operations to split the stream into its head and tail: `findFirst` and `skip`. Remember from [chapter 4](#) that after you call a terminal operation on a stream, it's consumed forever!

Lazy evaluation

There's an additional, more important problem: the static method `IntStream.concat` expects two instances of a stream, but its second argument is a direct recursive call to `primes`, resulting in an infinite recur-

Other formats available



Audiobook



restrictions on Java 8 streams such as no problematic and give your database-like ability to parallelize. Thus, the Java 8 design-

ers chose a sweet spot. Nonetheless, the more general features and models of streams from functional languages such as Scala and Haskell can be useful additions to your programming toolbox. What you need is a way to lazily evaluate the call to the method `primes` in the second argument of

concat. (In a more technical programming vocabulary, we refer to this concept as *lazy evaluation*, *nonstrict evaluation* or even *call by name*.) Only when you need to process the prime numbers (such as with the `limit` method) should the stream be evaluated. Scala (which we explore in [chapter 20](#)) provides support for this idea. In Scala, you can write the preceding algorithm as follows, where the operator `#::` does lazy concatenation (arguments being evaluated only when you need to consume the stream):

```
def numbers(n: Int): Stream[Int] = n #:: numbers(n+1)
def primes(numbers: Stream[Int]): Stream[Int] = {
  numbers.head #:: primes(numbers.tail filter (n => n % numbers.head != 0))
}
```

Don't worry about this code. Its only purpose is to show you an area of difference between Java and other functional programming languages. It's good to reflect for a moment about how the arguments are evaluated. In Java, when you call a method, all its arguments are fully evaluated immediately. But when you use `#::` in Scala, the concatenation returns immediately, and the elements are evaluated only when needed.

In the next section, we turn to implementing this idea of lazy lists directly in Java.

19.3.2. Your own lazy list

Java 8 streams are often described as being lazy. They're lazy in one particular aspect: a stream behaves like a black box that can generate values on request. When you apply a sequence of operations to a stream, these operations are merely saved up. Only when you apply a terminal operation to a stream is anything computed. This delaying has a great advantage when you apply several operations (perhaps a `filter` and a `map` followed by a terminal operation `reduce`) to a stream: the stream has to be traversed only once instead of for each operation.

In this section, you consider the notion of lazy lists, which are forms of a more general stream. (Lazy lists form a concept similar to stream.) Lazy lists also provide an excellent way of thinking about higher-order func-

Other formats available

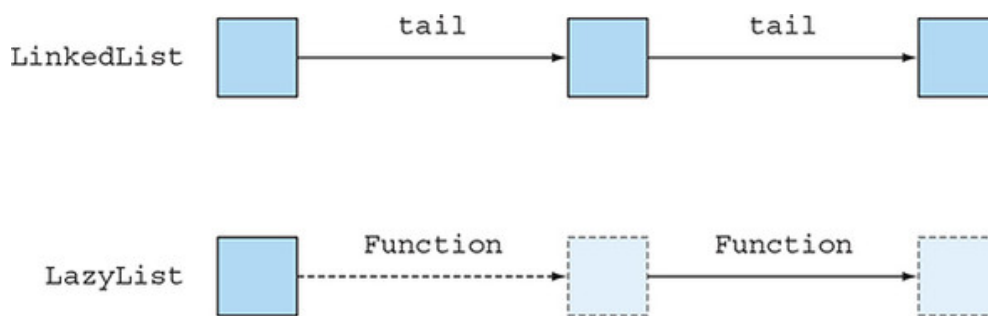


Audiobook



...ue in a data structure so that most of the
...ut when it's called (on demand), it can cre-
... [Figure 19.5](#) illustrates this idea.

Figure 19.5. Elements of a `LinkedList` exist (are spread out) in memory. But elements of a `LazyList` are created on demand by a Function; you can see them as being spread out in time.



Next, you see how this concept works. You want to generate an infinite list of prime numbers by using the algorithm we described earlier.

Creating a basic linked list

Recall that you can define a simple linked-list-style class called `MyLinkedList` in Java by writing it as follows (with a minimal `MyList` interface):

```
interface MyList<T> {
    T head();
    MyList<T> tail();
    default boolean isEmpty() {
        return true;
    }
}

class MyLinkedList<T> implements MyList<T> {
    private final T head;
    private final MyList<T> tail;
    public MyLinkedList(T head, MyList<T> tail) {
        this.head = head;
        this.tail = tail;
    }
    public T head() {
        return head;
    }
    public MyList<T> tail() {
        return tail;
    }
    public boolean isEmpty() {
        return false;
    }
}

class Empty<T> implements MyList<T> {
    public T head() {
        throw new UnsupportedOperationException();
    }
    public MyList<T> tail() {
        throw new UnsupportedOperationException();
    }
}
```

Other formats available



Audiobook



Now you can construct a sample `MyLinkedList` value as follows:

```
MyList<Integer> l =  
    new MyLinkedList<>(5, new MyLinkedList<>(10, new Empty<>()));
```

Creating a basic lazy list

An easy way to adapt this class to the concept of a lazy list is to cause the tail not to be present in memory all at once, but to have the `Supplier<T>` that you saw in [chapter 3](#) (you can also see it as being a factory with a function descriptor `void -> T`) produce the next node of the list. This design leads to the following code:

```
import java.util.function.Supplier;  
class LazyList<T> implements MyList<T>{  
    final T head;  
    final Supplier<MyList<T>> tail;  
    public LazyList(T head, Supplier<MyList<T>> tail) {  
        this.head = head;  
        this.tail = tail;  
    }  
    public T head() {  
        return head;  
    }  
    public MyList<T> tail() {  
        return tail.get();  
    }  
    public boolean isEmpty() {  
        return false;  
    }  
}
```

- **1 Note that tail using a Supplier encodes laziness, compared with method head.**

Calling the method `get` from the `Supplier` causes the creation of a node of the Lazy-List (as a factory would create a new object).

Now you can create the infinite lazy list of numbers starting at n as fol-

Other formats available



Audiobook

✕ tail argument of the `LazyList` constructor in the series of numbers:

```
public static LazyList<Integer> from(int n) {  
    return new LazyList<Integer>(n, () -> from(n+1));  
}
```

If you try the following code, you see that it prints 2 3 4. Indeed, the numbers are generated on demand. To check, insert `System.out.println` appropriately or note that `from(2)` would run forever if it tried to calculate all the numbers starting from 2:

```
LazyList<Integer> numbers = from(2);
int two = numbers.head();
int three = numbers.tail().head();
int four = numbers.tail().tail().head();
System.out.println(two + " " + three + " " + four);
```

Generating primes again

See whether you can use what you've done so far to generate a self-defining lazy list of prime numbers (something that you were unable to do with the Streams API). If you were to translate the code that was using the Streams API earlier, using the new `LazyList`, the code would look like something like this:

```
public static MyList<Integer> primes(MyList<Integer> numbers) {
    return new LazyList<> (
        numbers.head(),
        () -> primes(
            numbers.tail()
                .filter(n -> n % numbers.head() != 0)
        )
    );
}
```

Implementing a lazy filter

Unfortunately, a `LazyList` (more accurately, the `List` interface) doesn't define a `filter` method, so the preceding code won't compile! To fix this problem, declare a `filter` method, as follows:

```
public MyList<T> filter(Predicate<T> p) {
    return isEmpty() ?
        this :
        LazyList<> (head(), () -> tail().filter(p));
}
```

1

Other formats available



- 1 You could return `new Empty<>()`, but using `'this'` is as good and empty.

Your code compiles and is ready for use! You can calculate the first three prime numbers by chaining calls to `tail` and `head`, as follows:

```
LazyList<Integer> numbers = from(2);
int two = primes(numbers).head();
int three = primes(numbers).tail().head();
int five = primes(numbers).tail().tail().head();
System.out.println(two + " " + three + " " + five);
```

This code prints 2 3 5, which are the first three prime numbers. Now you can have some fun. You could print all the prime numbers, for example. (The program will run infinitely by writing a `printAll` method, which iteratively prints the head and tail of a list.)

```
static <T> void printAll(MyList<T> list){
    while (!list.isEmpty()){
        System.out.println(list.head());
        list = list.tail();
    }
}
printAll(primes(from(2)));
```

This chapter being a functional programming chapter, we should explain that this code can be neatly written recursively:

```
static <T> void printAll(MyList<T> list){
    if (list.isEmpty())
        return;
    System.out.println(list.head());
    printAll(list.tail());
}
```

This program wouldn't run infinitely, though. Sadly, it would eventually fail due to stack overflow because Java doesn't support tail call elimination, as discussed in [chapter 18](#).

Review

Other formats available



Audiobook

✕ technology with lazy lists and functions, using a data structure containing all the primes. What's the point of having a function on how to place functions inside data structures (because java 8 allows you to), and you can use these functions to create parts of the data structure on demand instead of when the structure is created. This capability may be useful if you're writing a game-playing program, perhaps for chess; you can have a data structure that

notionally represents the whole tree of possible moves (far too big to calculate eagerly) but that can be created on demand. This data structure would be a lazy tree, as opposed to a lazy list. We've concentrated on lazy lists in this chapter because they provide a link to another Java 8 feature, streams, which enabled us to discuss the pros and cons of streams compared with lazy lists.

There remains the question of performance. It's easy to assume that doing things lazily is better than doing things eagerly. Surely, it's better to calculate only the values and data structures needed by a program on demand than to create all those values (and perhaps more), as done in traditional execution. Unfortunately, the real world isn't so simple. The overhead of doing things lazily (such as the additional `Suppliers` between items in your `LazyList`) outweighs the notional benefit unless you explore, say, less than 10 % of the data structure. Finally, there's a subtle way in which your `LazyList` values aren't truly lazy. If you traverse a `LazyList` value such as `from(2)`, perhaps up to the 10th item, it also creates all the nodes twice, creating 20 nodes rather than 10. This result is hardly lazy. The issue is that the `Supplier` in `tail` is repeatedly called on each on-demand exploration of the `LazyList`. You can fix this problem by arranging for the `Supplier` in `tail` to be called only on the first on-demand exploration, with the resulting value being cached, in effect solidifying the list at that point. To achieve this goal, add a private `Optional<LazyList<T>> alreadyComputed` field to your definition of `LazyList` and arrange for the `tail` method to consult and update it appropriately. The pure functional language Haskell arranges that all its data structures are properly lazy in the latter sense. Read one of the many articles on Haskell if you're interested.

Our guideline is to remember that lazy data structures can be useful weapons in your programming armory. Use these structures when they make an application easier to program; rewrite them in more traditional style if they cause unacceptable inefficiency.

The next section deals with another feature of almost all functional programming languages except Java: pattern matching.

19.4. Pattern matching

Other formats available



Audiobook



...spect to what's generally regarded as functional) *pattern matching*, which isn't to be confused with pattern matching and regex. [Chapter 1](#) ended by observing that mathematics can write definitions such as

```
f(0) = 1
f(n) = n*f(n-1) otherwise
```

whereas in Java, you have to write an `if-then-else` or a `switch` statement. As data types become more complex, the amount of code (and clutter) needed to process them increases. Using pattern matching can reduce this clutter.

To illustrate, take a tree structure that you'd like to traverse. Consider a simple arithmetic language consisting of numbers and binary operations:

```
class Expr { ... }
class Number extends Expr { int val; ... }
class BinOp extends Expr { String opname; Expr left, right; ... }
```

Suppose that you're asked to write a method to simplify some expressions. `5 + 0` can be simplified to `5`, for example. Using our `Expr` class, `new BinOp("+", new Number(5), new Number(0))` could be simplified to `Number(5)`. You might traverse an `Expr` structure as follows:

```
Expr simplifyExpression(Expr expr) {
    if (expr instanceof BinOp
        && ((BinOp)expr).opname.equals("+")
        && ((BinOp)expr).right instanceof Number
        && ... // it's all getting very clumsy
        && ... ) {
        return (BinOp)expr.left;
    }
    ...
}
```

You can see that this code rapidly gets ugly!

19.4.1. Visitor design pattern

Another way to unwrap the data type in Java is to use the visitor design pattern. In essence, you create a separate class that encapsulates an algorithm to visit a specific data type.

Other formats available

 [Audiobook](#) 

...ing as input a specific instance of the data members. Here's an example. First, add the method `accept` to `BinOp`, which takes `SimplifyExprVisitor` as argument and passes itself to it (and add a similar method for `Number`):

```

class BinOp extends Expr{
    ...
    public Expr accept(SimplifyExprVisitor v){
        return v.visit(this);
    }
}

```

Now the `SimplifyExprVisitor` can access a `BinOp` object and unwrap it:

```

public class SimplifyExprVisitor {
    ...
    public Expr visit(BinOp e){
        if("+".equals(e.opname) && e.right instanceof Number && ...){
            return e.left;
        }
        return e;
    }
}

```

19.4.2. Pattern matching to the rescue

A simpler solution uses a feature called pattern matching. That feature isn't available in Java, so we're going to use small examples from the Scala programming language to illustrate pattern matching. The examples give you an idea of what could be possible in Java if pattern matching were supported.

Given data type `Expr` representing arithmetic expressions, in the Scala programming language (which we use because its syntax is closest to Java), you can write the following code to decompose an expression:

```

def simplifyExpression(expr: Expr): Expr = expr match {
    case BinOp("+", e, Number(0)) => e    // Adding zero
    case BinOp("*", e, Number(1)) => e    // Multiplying by one
    case BinOp("/", e, Number(1)) => e    // Dividing by one
    case _ => expr                       // Can't simplify expr
}

```

Other formats available



Audiobook



gives you an extremely concise and expres-
treelike data structures. Typically, this tech-
compilers or engines for processing business

rules. Note that the Scala syntax

```

Expression match { case Pattern => Expression ... }

```

is similar to the Java syntax

```
switch (Expression) { case Constant : Statement ... }
```

with Scala’s wildcard pattern `_` making the final case `_` play the role of default: in Java. The main visible syntactic difference is that Scala is expression-oriented, whereas Java is more statement-oriented. But for the programmer, the main expressiveness difference is the fact that Java patterns in case labels are restricted to a couple of primitive types, enumerations, a few special classes that wrap certain primitive types, and `Strings`. One of the biggest practical advantages of using languages with pattern matching is that you can avoid using big chains of `switch` or `if-then-else` statements interleaved with field-selection operations.

It’s clear that Scala’s pattern matching wins on ease of expressiveness over Java, and you can look forward to a future Java allowing more-expressive `switch` statements. (We make a concrete proposal for this feature in [chapter 21](#).)

In the meantime, we show you how Java 8 lambdas can provide an alternative way of achieving pattern-matching-like code in Java. We describe this technique purely to show you another interesting application of lambdas.

Faking pattern matching in Java

First, consider how rich Scala’s pattern-matching `match` expression form is. The case

```
def simplifyExpression(expr: Expr): Expr = expr match {  
  case BinOp("+", e, Number(0)) => e  
  ...
```

means “Check that `expr` is a `BinOp`, extract its three components (`op`-name, `left`, `right`), and then pattern-match these components—the first against the `String +`, the second against the variable `e` (which always matches), and the third against the pattern `Number(0)`.” In other words,

Other formats available



Audiobook



and many other functional languages) is multi-pattern matching with Java 8’s lambdas providing a simulation of pattern matching. In the preceding example, your simulation would express cases such as `BinOp(op, l, r)` or `Number(n)` but not `BinOp("+", e, Number(0))`.

First, we make a slightly surprising observation: now that you have lambdas, you could in principle never use `if-then-else` in your code. You could replace code such as `condition ? e1 : e2` with a method call, as follows:

```
myIf(condition, () -> e1, () -> e2);
```

Somewhere, perhaps in a library, you'd have a definition (generic in type `T`):

```
static <T> T myIf(boolean b, Supplier<T> truecase, Supplier<T> falsecase) {
    return b ? truecase.get() : falsecase.get();
}
```

The type `T` plays the role of the result type of the conditional expression. In principle, you can perform similar tricks with other control-flow constructs such as `switch` and `while`.

In normal code, this encoding would make your code more obscure because `if-then-else` captures this idiom perfectly. But we've noted that Java's `switch` and `if-then-else` don't capture the idiom of pattern matching, and it turns out that lambdas can encode (single-level) pattern matching—rather more neatly than the chains of `if-then-else`.

Returning to pattern-matching values of class `Expr` (which has two subclasses, `BinOp` and `Number`), you can define a method `patternMatchExpr` (again generic in `T`, the result type of the pattern match):

```
interface TriFunction<S, T, U, R>{
    R apply(S s, T t, U u);
}

static <T> T patternMatchExpr(
    Expr e,
    TriFunction<String, Expr, Expr, T> binopcase,
    Function<Integer, T> numcase,
    Supplier<T> defaultcase) {
    return
        e instanceof BinOp ?
            binopcase.apply(((BinOp)e).opname, ((BinOp)e).left,
                           ((BinOp)e).right) :
        e instanceof Number ?
            numcase.apply(((Number)e).val) :
            defaultcase.get();
}
```

Other formats available



The result is that the method call

```
patternMatchExpr(e, (op, l, r) -> {return binopcode;},
                  (n) -> {return numcode;},
                  () -> {return defaultcode;});
```

determines whether *e* is a *BinOp* (and if so, runs *binopcode*, which has access to the fields of the *BinOp* via identifiers *op*, *l*, *r*) or a *Number* (and if so, runs *numcode*, which has access to the value *n*). The method even makes provision for *defaultcode*, which would be executed if someone later created a tree node that was neither a *BinOp* nor a *Number*.

The following listing shows you how to start using `patternMatchExpr` by simplifying addition and multiplication expressions.

Listing 19.1. Implementing pattern matching to simplify an expression

```
public static Expr simplify(Expr e) {
    TriFunction<String, Expr, Expr, Expr> binopcase =
        (opname, left, right) -> {
            if ("+".equals(opname)) {
                if (left instanceof Number && ((Number) left).val == 0)
                    return right;
                if (right instanceof Number && ((Number) right).val == 0)
                    return left;
            }
            if ("*".equals(opname)) {
                if (left instanceof Number && ((Number) left).val == 1)
                    return right;
                if (right instanceof Number && ((Number) right).val == 1)
                    return left;
            }
            return new BinOp(opname, left, right);
        };
    Expr> numcase = val -> new Number(val);
    Expr> faultcase = () -> new Number(0);
    return patternMatchExpr(e, binopcase, numcase, faultcase);
}
```

Other formats available



Audiobook



```
Expr> numcase = val -> new Number(val);
Expr> faultcase = () -> new Number(0);
return patternMatchExpr(e, binopcase, numcase, faultcase);
```

- 1 Deals with a *BinOp* expression
- 2 Deals with the addition case
- 3 Deals with the multiplication case

- 4 Deals with a Number
- 5 A default case if the user provides an Expr that's not recognized
- 6 Applies pattern matching

Now you can call the `simplify` method as follows:

```
Expr e = new BinOp("+", new Number(5), new Number(0));
Expr match = simplify(e);
System.out.println(match);
```

1

- 1 Prints 5

You've seen a lot of information so far: higher-order functions, currying, persistent data structures, lazy lists, and pattern matching. The next section looks at certain subtleties that we've deferred to the end to avoid overcomplicating the text.

19.5. Miscellany

In this section, we explore two subtleties of being functional and of having referential transparency: one about efficiency and the other about returning the same result. These issues are interesting, but we place them here because the subtleties concern side effects and aren't conceptually central. We also explore the idea of *combinators*—methods or functions that take two or more functions and return another function. This idea has inspired many of the additions to the Java 8 API and more recently the Java 9 Flow API.

19.5.1. Caching or memoization

Suppose that you have a side-effect-free method `computeNumberOfNodes(Range)` that calculates the number of nodes inside a given range in a network with a treelike topology. Assume that the network never changes (that is, the structure is immutable), but that calling the method `computeNumberOfNodes` is expensive to calculate because the structure needs to be traversed recursively. You may want to calculate the results over and over. If you have referential transparency, you have a clever way to avoid this additional overhead. One standard so-

Other formats available



Audiobook

✗ g a cache (such as a `HashMap`) to the method

er consults the cache to see whether the (ar-

in the cache. If so, it can return the stored

result immediately. Otherwise, you call `computeNumberOfNodes`, but before returning from the wrapper, you store the new (argument, result) pair in the cache. Strictly speaking, this solution isn't purely functional

because it mutates a data structure shared by multiple callers, but the wrapped version of the code is referentially transparent.

In practice, this code works as follows:

```
final Map<Range,Integer> numberOfNodes = new HashMap<>();
Integer computeNumberOfNodesUsingCache(Range range) {
    Integer result = numberOfNodes.get(range);
    if (result != null){
        return result;
    }
    result = computeNumberOfNodes(range);
    numberOfNodes.put(range, result);
    return result;
}
```

Note

Java 8 enhances the Map interface (see [appendix B](#)) with a compute-If-Absent method for such use cases. You could use computeIfAbsent to write clearer code:

```
Integer computeNumberOfNodesUsingCache(Range range) {
    return numberOfNodes.computeIfAbsent(range,
                                         this::computeNumberOfNodes);
}
```

It's clear that the method computeNumberOfNodesUsingCache is referentially transparent (assuming that the method computeNumberOfNodes is also referentially transparent). But the fact that numberOfNodes has mutable shared state and that HashMap isn't synchronized^[3] means that this code isn't thread-safe. Even using (lock-protected) Hashtable or (concurrent-without-locking) ConcurrentHashMap instead of HashMap may not produce the expected performance if parallel calls are made to

Other formats available



multiple cores. There's a race condition between reading the value from the map and inserting the (argument, result) pair, which means that multiple processes might add to the map.

*This place is one where bugs breed. It's so easy to use `HashMap` and to forget the fact that the Java manual notes that it's not thread-safe (or not to care because **your** program is currently single-threaded).*

Perhaps the best thing to take away from this struggle is the fact that mixing mutable state with concurrency is trickier than you'd imagine. Functional-style programming avoids this practice except for low-level performance hacks such as caching. A second takeaway is that apart from implementing tricks such as caching, if you code in functional style, you never need to care whether another functional-style method that you call is synchronized, because you *know* that it has no shared mutable state.

19.5.2. What does “Return the same object” mean?

Consider again the binary tree example from [section 19.2.3](#). In [figure 19.4](#), variable `t` points to an existing `Tree`, and the figure shows the effect of calling `fupdate("Will", 26, t)` to produce a new `Tree`, which presumably is assigned to variable `t2`. The figure makes clear that `t` and all the data structures reachable from it aren't mutated. Now suppose that you perform a textually identical call in the additional assignment:

```
t3 = fupdate("Will", 26, t);
```

Now `t3` points to three more newly created nodes containing the same data as those in `t2`. The question is whether `fupdate` is referentially transparent. *Referentially transparent* means “equal arguments (the case here) imply equal results.” The problem is that `t2` and `t3` are different references, and therefore `(t2 == t3)` is `false`, so it looks as though you'll have to conclude that `fupdate` isn't referentially transparent. But when you're using persistent data structures that aren't to be modified, no logical difference exists between `t2` and `t3`.

We can debate this point at length, but the simplest adage is that functional-style programming generally uses `equals` to compare structured values rather than `==` (reference equality) because data isn't modified, and under this model, `fupdate` is referentially transparent.

19.5.3. Combinators

Other formats available



It's common and natural to write a higher-order function (or a method) that accepts, say, two functions and produces another function that somehow combines these functions. The term *combinator* generally is used for this idea. Much of the new Java 8 API is inspired by this idea, such as `thenCombine` in the `CompletableFuture` class. You can give this method two

CompletableFutures and a BiFunction to produce another CompletableFuture.

Although a detailed discussion of combinators in functional programming is beyond the scope of this book, it's worth looking at a couple of special cases to give you the flavor of how operations that take and return functions are a common and natural functional programming construct. The following method encodes the idea of function composition:

```
static <A,B,C> Function<A,C> compose(Function<B,C> g, Function<A,B> f)
    return x -> g.apply(f.apply(x));
}
```

This method takes functions *f* and *g* as arguments and returns a function whose effect is to do *f* first and then *g*. Then you can define an operation that captures internal iteration as a combinator. Suppose that you want to take data and apply function *f* to it repeatedly, *n* times, as in a loop. Your operation (call it *repeat*) takes a function, *f*, saying what happens in one iteration and returning a function that says what happens in *n* iterations. A call such as

```
repeat(3, (Integer x) -> 2*x);
```

returns *x* -> (2*(2*(2*x))) or equivalently *x* -> 8*x.

You can test this code by writing

```
System.out.println(repeat(3, (Integer x) -> 2*x).apply(10));
```

which prints 80.

You can code the *repeat* method as follows (noting the special case of a zero-trip loop):

```
static <A> Function<A,A> repeat(int n, Function<A,A> f) {
    return n==0 ? x -> x
           : compose(f, repeat(n-1, f));
}
```

Other formats available



- 1 Return the do-nothing identity function if *n* is zero.
- 2 Otherwise do *f*, repeated *n*-1 times, followed by doing it once more.

Variants of this idea can model richer notions of iteration, including having a functional model of mutable state passed between iterations. But it's time to move on. This chapter's role was to give you a summary of functional programming as the basis for Java 8. Many excellent books explore functional programming in greater depth.

Summary

- First-class functions are functions that can be passed as arguments, returned as results, and stored in data structures.
- A higher-order function takes one or more functions as input or returns another function. Typical higher-order functions in Java include `comparing`, `andThen`, and `compose`.
- Currying is a technique that lets you modularize functions and reuse code.
- A persistent data structure preserves the previous version of itself when it's modified. As a result, it can prevent unnecessary defensive copying.
- Streams in Java can't be self-defined.
- A lazy list is a more-expressive version of a Java stream. A lazy list lets you produce elements of the list on demand by using a supplier that can create more of the data structure.
- Pattern matching is a functional feature that lets you unwrap data types. You can view data matching as generalizing Java's `switch` statement.
- Referential transparency allows computations to be cached.
- Combinators are functional ideas that combine two or more functions or other data structures.

Other formats available



Audiobook

